

Pathfinding en Laberinto: BFS, DFS y A*

Introduccion: Problemas de busqueda

Un **problema de búsqueda** es la formalización de una situación donde querés pasar de un estado inicial a un estado objetivo, eligiendo acciones en cada paso.

Estado inicial: Primer estado del problema

Acciones posibles: Acciones posibles a ejecutar

Modelo de transición : Como tomas desiciones

Test de objetivo : Usado para verificar que se encontro la solucion

El resultado es un grafo de estados: La búsqueda consiste en explorar ese grafo sin conocerlo de antemano

Marco general: búsqueda en espacios de estados

Resolver un problema de búsqueda implica dos pasos: **modelarlo** y **explorarlo**. El desafío es que el espacio combinatorio de estados posibles suele ser demasiado grande para explorar. La clave está en *qué estado expandir primero*.
diferencian en el criterio de selección:

Algoritmo	Estructura	Criterio de expansión
BFS	Cola (Queue)	El más antiguo primero
DFS	Pila (Stack)	El más reciente primero
A*	Cola de prioridad	El de menor costo estimado $f(n)$

BFS — Búsqueda en Anchura (*Breadth-First Search*)

- Explora el grafo **nivel por nivel**, de forma horizontal antes de profundizar.
- Usa una **cola (FIFO)**: siempre expande el nodo más antiguo en la frontera.
- Debe mantener en memoria **todos los estados de la frontera actual**, lo que puede ser costoso.

Garantía: encuentra el **camino más corto** (mínima cantidad de pasos) cuando el costo de cada arista es uniforme.

Cuándo falla: en problemas con espacios de estados muy grandes, la memoria requerida crece exponencialmente con la profundidad.

DFS — Búsqueda en Profundidad (*Depth-First Search*)

- Explora el grafo **en profundidad**: sigue un camino hasta el final antes de retroceder.
- Usa una **pila (LIFO)**: siempre expande el nodo más reciente.
- Solo necesita guardar el **camino actual**, lo que la hace muy eficiente en memoria.

Limitaciones:

- **No garantiza optimalidad**: puede encontrar un camino más largo que el mínimo.
- **Puede no terminar**: en grafos con ciclos o espacios infinitos puede entrar en un loop sin llegar a la meta.

A* — Búsqueda Heurística

A* combina lo que ya se sabe con lo que se estima, eligiendo siempre el estado con menor costo total esperado.

Función de evaluación

$$f(n)=g(n)+h(n) \quad f(n) = g(n) + h(n)$$

$$f(n)=g(n)+h(n)$$

Componente	Significado
$g(n)$	Costo exacto acumulado desde el inicio hasta el nodo n
$h(n)$	Heurística: estimación del costo restante desde n hasta la meta
$f(n)$	Costo total estimado del camino que pasa por n

En cada paso, A* expande el nodo con **menor** $f(n)$ de todos los pendientes.

¿Por qué expande menos nodos que BFS?

BFS no tiene información sobre la dirección de la meta: expande en todas las direcciones por igual. A*, gracias a la heurística $h(n)$, **dirige la búsqueda hacia la meta**, evitando explorar regiones que claramente no llevan al objetivo.

Búsqueda informada vs. no informada

La diferencia fundamental entre los algoritmos de búsqueda está en si usan o no **información sobre el objetivo** para decidir qué nodo expandir primero.

BFS y DFS son búsqueda no informada. Ambos usan el test de objetivo únicamente para detectar cuándo terminarlo

A* es búsqueda informada. En cada paso consulta $h(n)$ para saber qué tan prometedor es cada nodo pendiente, y expande primero el que tiene menor costo total estimado $f(n) = g(n) + h(n)$. Esto le permite dirigir la búsqueda hacia la meta en lugar de expandir en todas las direcciones por igual.